

SE3090

Software Engineering Frameworks

Assignment 2

Software Engineering Frameworks

Assignment 2

Document control	Value
Submission type	Group assignment with individual viva
Maximum marks	100
Contribution to final grade	15%
Date prepared	2026-10-02
System under test	Universal SME Management Platform
Repository	SE3090_SE002-clean

Evidence integrity: This report uses current repository execution results. Docker-backed database tests, authenticated k6 scenarios, and live cross-client acceptance are explicitly marked pending where they were not executed in the local environment.

Executive Summary

This report evaluates the actual SE3090 integrated platform rather than a theoretical sample. The system consists of an ASP.NET Core 8 Web API, PostgreSQL persistence, a React web application, a Flutter mobile application, and a FastAPI Agentic AI service. Testing was selected around the highest risks: booking conflicts, authorization and tenant isolation, database integrity, client/API contracts, AI safety and approval enforcement, performance, and dependency security.

The fresh local execution baseline is strong: 595 backend tests passed, 194 React tests passed, 194 Agentic AI tests passed, and 154 Flutter tests passed. Flutter analysis and the React production build passed. The frontend dependency audit initially found four advisories; compatible updates were applied and the audit now reports zero vulnerabilities.

The report distinguishes passed, skipped, diagnostic, and pending evidence. Fifteen PostgreSQL/Testcontainers tests were skipped because Docker was unavailable. The deployed authenticated k6 scenarios and the complete Flutter-to-API-to-React workflow still require final demonstration evidence. These limitations are recorded to preserve assessment integrity and to make the remaining viva preparation explicit.

Overall assessment: The repository provides broad, meaningful automated coverage and a traceable evidence structure. The final Excellent mark depends on attaching the remaining live and environment-dependent evidence before CourseWeb submission.

1. Assignment Alignment

The assignment requires planned testing of the same integrated system, use of appropriate frameworks, normal/invalid/boundary/failure cases, current execution results, defect and retest evidence, performance and security testing, tool-generated evidence, and an individual viva contribution. This report and its companion test-case and evidence files address those requirements.

Assignment requirement	Evidence in this submission
Test plan	Scope, risks, objectives, tools, environment, responsibilities, schedule, and exit criteria in this report.
Test cases	Traceable API, database, React, Flutter, AI, integration, performance, and security cases in Section 5.
Actual results	Fresh execution totals and interpretation in Section 6, with passed/skipped/pending states.
Defects and retesting	Inventory timeout defect and npm security finding in Section 7.
Performance and security	k6 performance results and npm audit result in Section 8.
Individual viva	Ownership, commands, explanations, and likely questions in Section 11.

2. System Under Test

The platform is a multi-tenant SME management system. Clients communicate with the ASP.NET Core API; the API owns authentication, authorization, tenant scoping, validation, booking rules, migrations, audit-sensitive operations, and database access. The Agentic AI service is an internal FastAPI boundary called by the API and uses allow-listed tools. High-impact actions remain subject to deterministic validation and human approval.

Component	Technology	Testing focus
Backend/API	ASP.NET Core 8, EF Core, JWT, xUnit, Moq	HTTP behavior, validation, auth, roles, tenant isolation, booking and approval workflows.
Database	PostgreSQL/Supabase, EF Core migrations	Constraints, foreign keys, uniqueness, transactions, migration and query behavior.
Web	React 18, TypeScript, Vite, Redux Toolkit	Components, forms, protected routes, error states, AI workflow UI, build parity.
Mobile	Flutter, Dart, Riverpod, Dio	Widgets, validation, navigation, API refresh behavior, QR check-in and inventory.
Agentic AI	FastAPI, Pydantic, pytest, Gemini/Ollama boundary	Structured contracts, agent sequence, tool permissions, safety, failure recovery.

3. Test Strategy and Quality Risks

The strategy uses a layered approach. Unit and service tests isolate business rules; API integration tests exercise controllers and authorization boundaries; database integration tests use PostgreSQL/Testcontainers; React and Flutter

tests validate user-facing states; Agentic AI tests use deterministic model and backend fakes to make safety assertions repeatable; k6 and dependency audit provide non-functional evidence.

Risk	Control and evidence	Priority
Double booking or invalid availability	Conflict, overlap, capacity, adjacent-slot, cancellation, and rescheduling tests.	Critical
Cross-tenant data exposure	JWT, role, tenant, branch, and protected-route tests.	Critical
Unsafe AI action	Pydantic contract checks, allow-list tests, injection tests, deterministic approval gates.	Critical
Database corruption	PostgreSQL constraints, migration, transaction rollback, and relationship tests.	High
Client/API drift	React and Flutter API adapter, error-state, and contract-like tests.	High
Service saturation	k6 low-load pass and 20-VU diagnostic saturation run.	High
Dependency vulnerability	npm audit, remediation, and post-fix audit.	High

4. Environment, Tools, and Responsibilities

Area	Tool/framework	Responsible contribution
Backend/API	xUnit, Moq, WebApplicationFactory	Booking engine, authorization, controller and workflow tests.
Database	EF Core, PostgreSQL, Testcontainers	Schema integrity, constraints, transactions, migrations.
React	Vitest, Testing Library, jsdom	Protected routes, inventory, billing, booking and AI workflow UI.
Flutter	flutter_test, Dio adapter fakes	Login, booking, QR check-in, inventory and navigation.
Agentic AI	pytest, Pydantic, deterministic fakes	Golden cases, tool permissions, approval and safe failure.
Performance	k6	Public endpoint latency, failure rate, concurrency and saturation.
Security	npm audit plus authorization tests	Dependency remediation and behavioral authorization evidence.

The repository identifies the main contributors as Hasiru Chamika (booking and Agentic AI workflow), Oshadi (billing, payments, and Billing Copilot), and Hasaranga Abeyrathna (inventory, StockSense, mobile inventory, and supporting evidence). Final submission should attach the GitHub URL and per-member commit extracts.

5. Test Cases and Traceability

The following representative cases demonstrate normal, invalid, boundary, and failure-oriented design. The full companion matrix is stored in ``docs/assignment-2/TEST_CASES.md``.

ID	Scenario	Expected result	Result
API-001	Valid and invalid login	Valid user receives JWT; invalid input is rejected safely.	Passed
API-002	Wrong role, tenant, or token	401/403 response; no foreign data or mutation.	Passed
API-003	Overlap, adjacency, capacity, cancellation	Illegal conflict rejected; allowed boundaries accepted.	Passed
API-004	Disruption recovery approval lifecycle	Only accepted proposals apply; rejection is retained.	Passed
DB-001	Migration, FK, unique, CHECK constraints	Invalid writes fail and valid schema is created.	Pending Docker
WEB-001	Protected route roles	Redirect or render based on auth and role.	Passed
WEB-002	Inventory invalid then valid price edit	Validation blocks bad input; valid PUT refreshes row.	Passed
MOB-001	Login validation and token refresh	Validation shows; refresh retries once; refusal signs out.	Passed
AI-001	Structured schedule output	Contract parses and trace has required fields/order.	Passed
AI-002	High-impact action approval	Workflow remains AwaitingApproval.	Passed
AI-003	Prompt injection and tool permissions	No privilege escalation or unallow-listed tool.	Passed
E2E-001	Flutter request -> API -> React approval -> Flutter status	Same workflow state is visible across clients.	Pending live
PERF-001	Five virtual users on public endpoints	Failure <1%; p95 thresholds pass.	Passed
SEC-001	Frontend dependency audit	No high/critical advisories.	Passed

6. Test Execution Results

All results below were executed against the repository on 2026-10-02. They are not copied from historical usage logs.

Layer	Command/tool	Actual result	Status
Backend	dotnet test, Release, TRX	595 passed, 0 failed, 15 skipped, 610 total.	Passed with skips
React	npm run test	194 passed, 0 failed, 22 test files.	Passed
React build	npm run build	Navigation parity, subtype registry, TypeScript, Vite/PWA build passed.	Passed
Agentic AI	pytest tests/ -q	194 passed, 0 failed, 2 warnings.	Passed
Flutter analysis	flutter analyze	No issues found.	Passed
Flutter tests	flutter test	154 passed, 0 failed.	Passed
Security	npm audit --audit-level=high	0 vulnerabilities after remediation.	Passed
Performance	k6 deployed evidence	5 VUs: 0.99% failures and thresholds passed; 20 VUs: saturation and 429/5xx failures.	Diagnostic

Important interpretation: The 15 skipped backend tests are PostgreSQL/Testcontainers cases, not passes. Docker-backed execution is required for the strongest database evidence.

7. Defects, Fixes, and Retesting

Defect	Cause	Fix	Retest result
DEF-001: Inventory test timeout	Async inventory workflow exceeded Vitest default 5-second timeout under the full suite.	Added a justified test-specific 15-second timeout; assertions and production code unchanged.	Full React suite: 194/194 passed.
SF-001: npm advisories	Transitive brace-expansion, fast-uri, serialize-javascript, and undici advisories.	Ran compatible `npm audit fix`, updating the lockfile.	npm audit: 0 vulnerabilities.

The frontend timeout was first reproduced in the full suite, then the single test was rerun and passed in 702 ms with a longer timeout. The complete suite and production build were rerun after the change. This establishes a before/fix/retest chain suitable for the defect report.

8. Non-Functional Testing

8.1 Performance testing

The k6 script uses ramping virtual users and measures health/database round trip, business directory, and public booking catalog endpoints. At five virtual users, the recorded run produced 1,014 requests, a 0.99% failure rate, and passing thresholds. At twenty virtual users, the deployment exceeded thresholds, with increased latency and HTTP 429/5xx responses. This is a meaningful capacity result: the test found the free-tier edge limit instead of hiding it.

Authenticated-read and Agentic AI latency scenarios were not included in the historical run because approved demo credentials were unavailable. They must be executed before final submission and added to the evidence folder.

8.2 Security testing

Behavioral security tests cover authentication, role authorization, tenant isolation, approval boundaries, prompt-injection resistance, and tool allow-lists. The frontend dependency scan initially found four advisories; remediation was applied and the final audit returned zero vulnerabilities. The exposed credential file `scripts/pass.txt` must be rotated and excluded from the final submission.

9. Integrated Workflow Demonstration

The repository contains an auditable disruption recovery workflow. The supplied Mirissa Jetliner seed script creates four stranded bookings and ten guests on the Dawn Departure and leaves a Morning Cruise option for recovery. The intended demonstration is: seed the data, sign in, open Bookings -> Disruption Recovery, select the failed departure and tomorrow dates, run the recovery plan, inspect affected guests and safety checks, approve selected proposals, and verify the resulting booking state and audit trace.

Repository unit and API integration tests already cover the approval lifecycle and the React disruption recovery page covers the UI states. A final live cross-platform acceptance must still be captured for E2E-001: customer/mobile request, API orchestration, React manager approval, API application, and mobile status refresh. This report deliberately does not claim that live flow as completed without its screenshot and trace ID.

```
psql "$DATABASE_URL" -f scripts/seed-mirissa-jetliner-disruption.sql
```

10. Rubric Evidence Matrix

Rubric criterion	Excellent evidence already prepared	Final attachment needed
Testing strategy and coverage	Risk-based plan, layered tools, representative normal/invalid/boundary/failure cases.	Attach this report and full test-case matrix.
Integrated and non-functional testing	API approval lifecycle, k6 performance, security audit and behavioral security tests.	Live E2E screenshots and authenticated k6 output.
Results, defects, documentation	Fresh totals, defect/retest chain, report, companion documents and runner.	TRX, JUnit, coverage and screenshot attachments.
Tool/framework demonstration	Commands and ownership mapped to xUnit, Vitest, pytest, Flutter, k6.	Each student demonstrates one owned test live.
Test implementation and execution	Meaningful assertions across all clients and AI safety paths.	Git commit links and modified-test demonstration.
Results, defects and retesting	DEF-001 and SF-001 include reproduction, fix and retest.	Attach raw before/after output.
Technical contribution	Ownership table and source paths are documented.	GitHub URL and `git log --author` extracts.
Viva and understanding	Viva guide contains explanations and likely questions.	Rehearse without exposing secrets.

11. Individual Viva Preparation

Each student should select one test they personally implemented or materially changed, run it live, explain the risk, identify the assertion, and modify one input to show that the test is meaningful. Students should be ready to explain what is mocked, what is real, why a skipped test is not a pass, how the AI approval gate prevents unsafe action, and what the 20-VU k6 result means.

Student focus	Recommended demonstration	Key explanation
Hasiru	Booking conflict or disruption approval xUnit test plus AI safety case.	Why only validated, approved proposals can be applied.
Oshadi	Billing threshold or dynamic form React/xUnit test.	Why deterministic billing limits cannot be changed by model output.
Hasaranga	Inventory price edit or Flutter inventory test plus npm audit.	How invalid input, valid save, API payload, and security remediation are verified.

12. Reproducibility and Evidence Inventory

The companion directory `docs/assignment-2/` contains the Markdown test plan, test cases, defect log, execution summary, rubric matrix, and viva guide. The script `tests/assignment2/run-baseline.ps1` reruns the local gates and writes logs under `tests/evidence/assignment-2/`. The Agentic AI JUnit result generated during execution is stored at `tests/evidence-agent-pytest.xml`. Backend TRX output is generated under `backend/SmeBackend.Tests/TestResults/`.

```
dotnet test backend/SmeBackend.Tests/SmeBackend.Tests.csproj --configuration Release --logger
"trx;LogFileName=assignment-backend.trx"

Set-Location frontend; npm ci; npm run test; npm run build; npm audit --audit-level=high

Set-Location agentic-ai-service; python -m pytest tests/ -q
--junitxml=../tests/evidence/assignment-2/agent-tests.xml

Set-Location mobile/sme_mobile; flutter pub get; flutter analyze; flutter test
```

Final attachments should include raw test output, coverage reports where available, k6 JSON/Markdown summaries, sanitized screenshots, GitHub repository and commit evidence, lecturer group-size approval, and the declared AI usage record.

13. Final Submission Checklist

- Attach this report as PDF after reviewing the generated DOCX.
- Attach completed test-case document with expected, actual, and Pass/Fail status.
- Attach defect/bug report with reproduction and retesting evidence.
- Run Docker-backed PostgreSQL tests and attach the TRX result.
- Run authenticated and Agentic AI k6 scenarios and attach raw output.

- Demonstrate the complete Flutter -> API -> React -> API -> Flutter workflow and attach screenshots/trace ID.
- Generate and attach coverage reports and CI artifact links.
- Add GitHub URL, per-member commits, lecturer group-size approval, and AI declaration.
- Rotate the exposed credential in `scripts/pass.txt`; never submit or publish that credential.
- Ensure every group member can explain and reproduce their own testing contribution during the viva.

Declaration of evidence status: This report is complete as a repository-grounded testing report. Any item labelled Pending requires execution and attachment before the final CourseWeb submission; no pending result has been fabricated.

Appendix A - Evidence File Map

Evidence	Location
Test plan and scope	docs/assignment-2/TEST_PLAN.md
Test cases	docs/assignment-2/TEST_CASES.md
Defect and retest record	docs/assignment-2/DEFECT_LOG.md
Execution summary	docs/assignment-2/EXECUTION_SUMMARY.md
Rubric traceability	docs/assignment-2/RUBRIC_EVIDENCE_MATRIX.md
Viva preparation	docs/assignment-2/VIVA_GUIDE.md
Automated runner	tests/assignment2/run-baseline.ps1
Backend tests	backend/SmeBackend.Tests/
React tests	frontend/src/**/*.*.test.*
Flutter tests	mobile/sme_mobile/test/
Agent tests	agentic-ai-service/tests/
Performance script/results	tests/performance/
CI pipeline	.github/workflows/ci.yml